# A Microservices-Based Home Energy Tracker for Event-Driven Monitoring, Deduplicated Alerting, and ML-Based Anomaly Detection

Rohit Shrivastava[1], Harsh Gupta[2], Anshika Rai[3], Syamasudha Veeragandham[4]
[1,2,3,4]School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014
Email: [1]rohit.lalbahadur2023@vitstudent.ac.in, [2]harsh.gupta2023@vitstudent.ac.in, [3]anshika.rai2023@vitstudent.ac.in, [4]syamasudha.v@vit.ac.in
[4]Corresponding author

## Abstract

Residential energy monitoring requires a traceable path from telemetry admission to storage, analysis, and controlled notification. This paper presents a local Home Energy Tracker implemented as event-driven microservices with gateway-mediated access control, device-bound telemetry tokens, Apache Kafka, InfluxDB, MySQL, deterministic insights, a duplicate-aware alert path, a legacy Random Forest inference service, and service observability. The repository classifier was previously evaluated on a 5,000-record labelled CSV and reproduced offline; its held-out F1 score of 0.902439 is therefore historical-dataset evidence, not REFIT, field, or household-level accuracy. A Wokwi-compatible request confirms that the inference interface can load the model and return a response, but it is only a smoke test. For data provenance, five official REFIT cleaned files were checksum-verified and the H1/H2/H4 primary cohort was transformed into 468,847 interval-energy records, totalling 386.398411389 kWh under the documented actual-delta sample-hold rule. H3/H11 remain a separate solar-affected sensitivity cohort. End-to-end REFIT ingestion, replay, alert-deduplication validation, and REFIT anomaly-accuracy evaluation are pending. The contribution is thus an implemented, reproducibility-oriented system with explicit evidence limits.

**Keywords:** Home energy management, microservices, Apache Kafka, anomaly detection, REFIT, reproducibility, IoT telemetry, alert deduplication.

## 1. Introduction

### 1.1 Residential Telemetry as a Data-to-Decision Chain

A residential monitoring application receives more than a numerical consumption value. It must determine whether a message belongs to an authorised device, whether the payload is structurally usable, where the accepted record is retained, which downstream calculation sees it, and whether a later notification can be related to the same household context. A dashboard can display a recent value without answering these questions. In contrast, a

monitoring system has to keep the telemetry path sufficiently explicit that an unexpected total, absent alert, or rejected reading can be located at the appropriate boundary.

The distinction is especially important for simulator-compatible input. The present project accepts software-generated telemetry that follows the application contract, but that contract alone does not establish the physical origin, calibration, sampling behaviour, or billing meaning of a reading. The system is therefore presented as a local software prototype that demonstrates controlled data handling. Physical smart-meter and smart-plug studies remain relevant as measurement contexts, but they do not make a simulated request equivalent to a calibrated field instrument [12], [13], [15], [20].

The operational difficulty is not confined to the first HTTP request. Residential readings may be malformed, submitted with an unauthorised device identity, delayed while downstream consumers are unavailable, or interpreted under a unit convention that is not expressed by the payload itself. Each of those cases has a different corrective action. The study consequently treats admission, event publication, persistence, inference, and notification as separate responsibilities rather than using a single endpoint response as evidence that the whole data path behaved correctly.

### 1.2 System Challenges and Architectural Rationale

The implemented architecture separates browser-facing control activity from the telemetry data path. Keycloak and the API Gateway mediate user-facing access, whereas the ingestion route can admit a machine-originated record only after device-token checking. This separation reflects a practical difference between a user asking to view household data and a meter-like client sending a single measurement. It also makes a household identifier, device identifier, and caller identity distinct entities rather than assuming that one number proves the others.

After admission, Kafka provides an asynchronous boundary between telemetry publication and downstream processing. In this design, decoupling is a design rationale: it allows the Ingestion Service to publish an accepted event without embedding time-series writes, alert decisions, and inference logic in the same request path. It does not, by itself, demonstrate an increase in throughput or fault tolerance. The project consequently reports code-level retry and dead-letter routing separately from runtime recovery measurements, and it does not claim exactly-once processing or complete record-level correlation across every service.

The system also distinguishes time-series values from relational state. InfluxDB holds timestamped usage values, while MySQL is used for user, device, and alert records. This allocation allows a usage query to work over time-stamped values while ownership and alert state remain relational. The choice is aligned with the modular energy-management reasoning in [1]–[3] and [17], but the paper does not infer database scalability or billing accuracy from the choice of technologies.

### 1.3 Model Evidence and Data Provenance

An anomaly score cannot be interpreted independently of the data that produced it. Its meaning depends on the source of the measurements, the construction of the predictor variables, the definition of the abnormal class, the split between training and evaluation periods, and the decision rule used to turn a probability into a label. The legacy classifier in this repository can be reproduced under its documented local protocol, but the supplied CSV does not establish an external source, licence, household identity, label-generation process, or causal construction of Avg_Past_Consumption. Reproducibility of that experiment is therefore valuable but narrower than a claim of generalisation.

The study deliberately separates four evidence classes: implemented source behaviour, dated local runtime observations, deterministic offline reconstruction of the legacy supervised experiment, and REFIT source/preprocessing evidence. These classes answer different questions. A local HTTP response can show that an interface executed; a repeated model artifact can show that a historical procedure is reproducible; and a verified checksum can show that a source file was retained correctly. None of them independently establishes anomaly accuracy on a new household.

### 1.4 Scope, Contribution, and Paper Organisation

The central research problem is to describe a residential energy-monitoring prototype without conflating its implemented software, historical model evidence, simulator-compatible checks, and recorded-household data preparation. The contribution is an evidence-aware account of a gateway-mediated, event-driven architecture; a reproducible reconstruction of a legacy Random Forest baseline; and a checksum-traceable REFIT preprocessing procedure for a future isolated evaluation. The paper does not claim that REFIT data have already passed through the live ingestion route, that the classifier is accurate on REFIT, or that alert behaviour has been proven under a REFIT replay.

Section 2 situates the work within energy-management, monitoring, anomaly-detection, and dataset-provenance literature. Section 3 describes the implementation and the separated evidence streams. Section 4 interprets the historical baseline, dated local checks, and REFIT preprocessing results at their supported scope. Section 5 identifies the technical limitations and the acceptance-gated future evaluation, and Section 6 concludes the study.

Residential energy monitoring is more than a display of consumption: useful systems must admit telemetry, preserve an audit trail, derive operational insights, and issue bounded notifications without being mistaken for calibrated utility metering. Microservice architectures provide a practical way to separate these responsibilities into inspectable components [1], [2].

This study concerns a local Home Energy Tracker, not a calibrated utility-metering product. Simulator-compatible telemetry is used to examine software-path execution, reproducibility,

and data preparation; it does not provide physical sensor calibration or utility-billing accuracy. Physical-meter prototypes therefore represent a separate validation category [13], [15].

An event-driven design decouples telemetry admission from downstream aggregation, storage, inference, and alert processing. Recent energy-management architectures use modular services and asynchronous communication for this separation [1], [3]. Their deployment conditions are not treated as numerical baselines for the present local prototype.

In the present system, Keycloak supplies JWT-, role-, and ownership-based control for user-facing requests, while simulator-compatible telemetry is admitted through a device-bound token. These mechanisms define an admission boundary before event processing begins.

Validated telemetry is published to Kafka. The Usage Service records time-stamped usage and windowed aggregates; InfluxDB stores time-series usage, while MySQL stores household, device, and alert state. This separation makes the processing path inspectable and keeps relational state distinct from time-series data.

The implementation also exposes deterministic bill forecasting, appliance ranking, standby detection, saving tips, and related insights. Their usefulness depends on being traceable to admitted telemetry, so this study prioritises record handling and evidence boundaries over dashboard presentation alone.

Smart-meter anomaly research links a model result to the measurement source, label definition, feature construction, and evaluation protocol. Studies of daily activities, faults or misuse, dynamic anomaly detection, and real-time AI pipelines therefore provide context for interpreting—not inflating—the evidence of the present classifier [4], [8], [10], [19].

The installed Random Forest is a legacy repository artefact. Although its local reconstruction follows a documented protocol, the historical labelled CSV has incomplete source, licence, label-generation, and causal-feature provenance. Its metrics are consequently treated as reproducibility evidence rather than evidence of generalisation.

REFIT provides a more thoroughly documented household-data source for future evaluation. The descriptor and cleaned-data records establish measurement provenance, but aggregate-power preparation and anomaly-label alignment still require verification before the data can support end-to-end anomaly claims [5]–[7].

This paper documents the implemented architecture, a reproducible historical baseline, and checksum-verified REFIT preparation. It distinguishes those completed activities from isolated provisioning, replay, alert-deduplication validation, and REFIT anomaly-accuracy evaluation, which remain future validation work.

### 1.5 Research Objectives

1. Objective 1: Design a secure microservices data pipeline with telemetry validation, device-token checking, inspectable service boundaries, and invalid-message handling.
2. Objective 2: Integrate Kafka with the microservices to support event-driven telemetry processing, baseline inference, alert handling, and observability for failures, replay, consumer lag, and tracing.
3. Objective 3: Establish a reproducible Random Forest baseline by recording the legacy features, random seed, and chronological split, while clearly separating offline metrics from real-world validation.
4. Objective 4: Prepare and integrity-check REFIT records, document data-quality outcomes, and define evaluation steps for future end-to-end anomaly-detection validation.

Objective-to-evidence alignment: Objective 1 is addressed by the gateway and device-token admission boundary, with independent security testing pending. Objective 2 is addressed by the Kafka-integrated service design and monitoring components, with replay, lag, fault, and recovery tests pending. Objective 3 is supported by deterministic reproduction of the historical baseline, with external and household-level evaluation pending. Objective 4 is supported by REFIT integrity checking, cohort selection, and normalisation, with ingestion, label alignment, replay, and accuracy evaluation pending.

**Objective Scope and Evaluation Boundary**

Objective 1 addresses the admission boundary: a telemetry system cannot safely associate a reading with a household unless the request contract, caller category, and device association are checked before event publication. The implemented approach uses gateway-mediated user context for interactive requests and a device-bound token for machine telemetry. Available evidence includes the implementation and selected dated local authorisation checks; independent security evaluation and a credential lifecycle remain outside the study.

Objective 2 addresses processing visibility. Kafka, time-series storage, relational state, alert handling, and observability components make the intended stages visible, while the source includes bounded retry and dead-letter routing for a failed time-series write. Dated local checks and component tests support selected parts of the path. They do not yet establish sustained consumer-lag behaviour, cross-service event reconciliation, or recovery under the complete REFIT workload.

Objective 3 addresses reproducible interpretation of the legacy model. The study records the data split, features, configuration, threshold protocol, and exact reconstructed results so that a historical score is not treated as an opaque model claim. The evidence is sufficient to show reproduction of the available CSV experiment, but not to resolve the source and feature-provenance limitations or to support new-household accuracy.

Objective 4 addresses preparation for a recorded-household experiment. REFIT files are verified, cohort selection is declared, interval energy is estimated with actual time

differences, and derived artifacts are reproduced byte-for-byte. The objective intentionally stops before live ingestion, device provisioning, label alignment, and model evaluation. Those remaining activities are described as staged validation requirements rather than as completed work.

## 2. Related Work

### 2.1 Microservices and Service Separation for Energy Management

Recent energy-management research provides several reasons to isolate monitoring responsibilities. Rovetto et al. [1] present residential energy-efficiency monitoring through services, while Motta et al. [2] describe a cloud Home Energy Management System model that differentiates ingestion, operational, and analytical environments. These publications support a conceptual decomposition in which a reading should be handled differently from an operational request or an analytical result. They do not prescribe the exact services, storage systems, or security boundary used in the present local implementation.

Veloso et al. [3] extend the modular argument into smart-grid settings by combining hybrid communication with a microservices solution. Zhu and Yue [17] likewise use technologies including Docker, Kafka, and InfluxDB in a smart-city energy-management setting. Their work demonstrates that the same categories of concerns can be separated in larger installations. However, published deployment outcomes from different locations, communication assumptions, and workloads cannot be transferred directly to this residential prototype. The present study therefore uses them to justify architectural roles, not to establish a numerical benchmark.

Across these studies, a modular architecture is useful only if boundaries are inspectable. In the present system, the Gateway controls public admission; Ingestion publishes telemetry; Usage stores and aggregates consumption; the inference service produces a model response; Alert Service applies notification policy; and monitoring components observe services. That decomposition reduces coupling in the design, but a complete trace from input to final effect still requires runtime evidence. In particular, the core energy event does not by itself supply a durable end-to-end correlation identifier, so the current architecture should not be described as providing complete record-level tracing.

The literature therefore establishes service separation as a viable architectural pattern, while leaving an application-specific question about evidence: which transitions have actually been observed and which remain design intent? This question motivates the following themes on measurement, anomaly interpretation, and source preparation.

Rovetto et al. [1] use a microservices architecture for residential energy-efficiency monitoring. Their work motivates the separation of monitoring responsibilities; the present

study applies that principle to a local prototype and does not reuse the reported deployment outcomes as performance evidence.

Motta et al. [2] organise home-energy functions around ingestion, operational, and analytical environments. This layered view helps frame the present gateway, event-processing, storage, inference, and alert components, while the present study makes no claim to replicate the referenced cloud-scale setting.

Veloso et al. [3] combine hybrid communication with a modular microservice platform for smart-grid energy management. Their work demonstrates why communication, processing, and service responsibilities should be separated, whereas the present study confines its claim to local service implementation and leaves communication-scale and fault-tolerance evaluation outside the reported evidence.

A common implication of these architectures is that a high-level system diagram is not an evaluation protocol. A service may be implemented, started, or reachable without proving that a particular telemetry record is delivered exactly once, persisted under the intended timestamp, or carried into the alert decision that a user sees. This distinction guides the evidence labels used throughout the present paper.

At the anomaly-detection level, Hernández et al. [4] connect smart-meter measurements with a defined evaluation context for activity anomalies. That connection is important here because the legacy CSV's labels and provenance are incomplete: its reported metrics can reproduce a historical experiment, but cannot establish household-level anomaly accuracy.

**2.2 Smart-Meter Anomaly Detection and Evaluation Context**

Hernández et al. [4] relate smart-meter observations to anomalies in daily activity. The important lesson for this work is methodological rather than numerical: the meaning of an anomaly depends on the measurement setting and the target definition. A classifier score can appear strong under a documented experiment yet be inappropriate for a different household signal, label ontology, or feature set. This is directly relevant to the present legacy CSV, whose labels and source history are not fully documented.

Kulkarni et al. [8] discuss faults, billing errors, and misuse in a smart-meter anomaly setting. That application context supports the need for careful admission and alerting, but the publication's data and labels are not a substitute for the repository's historical CSV or for REFIT. Maitra et al. [10] propose a convolutional autoencoder with a dynamic threshold; its design illustrates a different way to treat temporal variation, but it is an arXiv preprint and is not used as an empirical comparator for the fixed Random Forest retained here.

Liu and Hsiao [12] combine smart-plug monitoring with regression-based anomaly detection. Shuaibu et al. [19] describe appliance monitoring and anomaly inference over DDS middleware. Both extend beyond the evidence in this paper: smart-plug systems make particular physical measurement assumptions, and appliance-level pipelines define a

different target from a whole-house aggregate energy record. Their results cannot be ranked against the historical Random Forest score without a common dataset, labels, split, and feature contract.

The existing classifier uses a supervised Random Forest rather than an autoencoder, a regression model, or a multi-task network. Its contribution in this study is not a new learning algorithm. Instead, the work makes the legacy configuration and its reproducibility boundary explicit: a 70/15/15 chronological split, validation-based threshold selection, a fixed feature set, and a separated held-out test. This allows the historical result to be understood at the correct level of evidence.

The studies in this theme collectively show that model type alone does not define validity. Measurement source, feature availability, label semantics, and evaluation protocol are equally decisive. The present paper uses that conclusion to avoid treating a Wokwi-compatible inference response or a REFIT checksum as an anomaly-detection result.

Kulkarni et al. [8] frame anomaly detection in smart electric meters around faults, billing errors, and misuse. Their application context is relevant, but its data, labels, and method do not constitute a benchmark for the fixed legacy Random Forest used in this study.

Maitra et al. [10] describe a convolutional-autoencoder approach with a dynamic threshold for real-time meter data. It provides an alternative modelling strategy, but it is an arXiv preprint and is not used as a direct performance comparator for the historical Random Forest.

IoT-monitoring studies also focus on user-facing and physical-meter settings. Karthika et al. [11] provide smart-energy-monitoring context; Liu and Hsiao [12] report smart-plug-based home energy management with regression-based anomaly detection; and Hidayat and Al-Faris [13] use an ESP32/PZEM-004T monitoring prototype. These physical-monitoring contexts underscore why the Wokwi-compatible request in this study must not be presented as calibrated meter evidence.

### 2.3 IoT Monitoring, Physical Measurement, and Notification Context

Karthika et al. [11] provide IoT energy-monitoring context, while Hidayat and Al-Faris [13] use an ESP32/PZEM-004T prototype to obtain physical electrical parameters. Hossain and Hassan [15] similarly report a real-time meter configuration. De Sousa and Rabelo [20] include residential consumption and power-quality considerations. These studies are valuable because they make the measurement layer visible; they also highlight why an application payload generated by a simulator should not be presented as a physical electrical measurement.

Notification is another boundary that should be separated from the predictive score. A threshold crossing, model anomaly, stored alert, e-mail send attempt, and final recipient delivery are related but not identical events. The present system includes threshold and

anomaly pathways plus a 15-minute duplicate-suppression policy. It is a local design and a selectively tested implementation, not a replacement for the operational guarantees that may be needed in a deployed utility or clinical-like notification system.

Sirisumrannukul et al. [14] integrate ANNs and particle-swarm optimisation in a cooling-management microservice platform, making optimisation and autonomous control central to their contribution. Hossain and Hassan [15] report a real-time smart-energy-meter system, and Zhu and Yue [17] describe smart-city energy management using technologies that include Docker, Kafka, and InfluxDB. These studies provide complementary application and architectural context, but their deployment settings and outcomes are not transferred to the present local tracker.

Shuaibu et al. [19] present a smart-metering pipeline that uses DDS middleware and multi-task neural inference for appliance monitoring and anomaly detection. De Sousa and Rabelo [20] combine residential consumption monitoring with power-quality analysis. Both works illustrate broader operational goals than those demonstrated here: the present system does not claim appliance-level inference, power-quality classification, field calibration, or comparable real-time performance.

REFIT is treated separately from these research studies because it is a dataset and data-provenance resource. Murray et al. [5] document the household measurements, while the University of Strathclyde and Zenodo records describe cleaned data access [6], [7]. The present work uses these sources for checksum verification, cohort selection, and repeatable interval-energy preparation rather than as a source of validated aggregate anomaly labels.

**2.4 Household Data, Preprocessing, and Reproducibility**

REFIT is different from the architecture and model studies because it is a documented household-data resource. Murray et al. [5] describe the measurement study, and the University of Strathclyde and Zenodo records [6], [7] provide the cleaned-data provenance used in this project. The dataset is valuable because it exposes household aggregate active power, paired human-readable and Unix timestamps, and a documented distribution. It should not be represented as a ready-made aggregate anomaly benchmark merely because it contains rich time-series data.

For a later event-stream experiment, source integrity, timestamp ordering, the treatment of gaps, and the power-to-energy convention all affect the values presented to the application. The current work therefore verifies the local files against published MD5 values and records a deterministic conversion rule. This approach establishes a repeatable derived input, but it cannot resolve the semantic status of every source annotation or provide aggregate anomaly labels without a separate alignment protocol.

Taken together, the reviewed work establishes the value of modular energy architectures, IoT monitoring, and anomaly-analysis methods. It also shows that evidence is tied to a

particular data source, measurement system, and protocol. The present study therefore contributes an evidence-aware local architecture and a reproducible data-preparation path, while retaining clear boundaries around its untested runtime and external-validation stages.

### 2.5 Critical Synthesis

The literature supports three linked observations. First, modular architectures can make energy-monitoring responsibilities separable, but their production or city-scale outcomes do not transfer automatically to a local composition. Second, anomaly-detection outcomes are conditional on a dataset, labels, and evaluation design; different algorithms cannot be compared by a score alone. Third, physical monitoring systems and household datasets impose measurement and provenance assumptions that simulator-compatible telemetry does not remove. The present study uses these observations to frame an implementation and reproducibility contribution rather than an unqualified performance claim.

Table I is intentionally compact because the single-column layout cannot accommodate a reliable seven-column literature matrix without reducing readability. Its existing columns jointly capture the research problem, method or setting, reported contribution, and evaluation boundary. The surrounding themed discussion provides the context that a table cannot carry safely.

The comparison in Table I summarises representative studies by objective, approach, contribution, and the evaluation boundary relevant to the present study.

**Table I. Thematic comparison of architecture, monitoring, anomaly-detection, and data-provenance studies**

| Reference and year | Research objective | Method, architecture, or setting | Reported contribution | Relevant boundary and relationship to this study |
|---|---|---|---|---|
| [1], 2024 | Residential energy-efficiency monitoring | Microservices architecture | Separates monitoring responsibilities into services. | Motivates component separation; its deployment findings are not reused as benchmarks. |
| [2], 2025 | Home energy-management architecture | Cloud HEMS conceptual model | Organises ingestion, operational, and analytical environments. | Supports the architectural framing; the present study does not claim its cloud-scale results. |
| [3], 2026 | Smart-grid energy management | Hybrid communication with microservices | Presents and validates a modular smart-grid platform in its own setting. | Supports decoupled design; communication and deployment metrics are not transferred. |
| [4], 2024 | Anomaly detection from smart-meter data | Daily-activity analysis with a real household meter | Links anomaly interpretation to a documented measurement and evaluation context. | Reinforces the need to keep data, labels, and protocol explicit; task differs from the legacy classifier. |
| [8], 2024 | Fault and misuse identification | Smart-meter data analytics and machine learning | Uses anomaly detection for faults, billing errors, and misuse context. | Relevant anomaly application context; labels, features, and data are not those of the present study. |
| [10], 2024 (preprint) | Real-time energy-anomaly detection | Convolutional autoencoder with dynamic threshold | Presents an alternative model using meter data and an adaptive threshold. | A preprint and not a direct baseline; no performance comparison is claimed with the legacy Random Forest. |

| Reference and year | Research objective | Method, architecture, or setting | Reported contribution | Relevant boundary and relationship to this study |
|---|---|---|---|---|
| [11], 2025 | Smart energy monitoring | IoT-oriented monitoring system | Provides application-level monitoring context. | Used only for monitoring context; source-level empirical detail is limited for a direct comparison. |
| [12], 2026 | Home energy management and anomaly detection | Smart plugs, Zigbee/MQTT, and regression analysis | Reports an IoT HEMS with predictive and anomaly-analysis functions. | Shows a physical smart-plug approach; it is not a benchmark for simulator-compatible telemetry. |
| [13], 2025 | Household consumption monitoring | ESP32 and PZEM-004T prototype | Implements physical measurement of electrical parameters and IoT reporting. | Clarifies the distinction between physical-meter validation and the present Wokwi-compatible interface test. |
| [14], 2024 | Cooling-energy conservation | ANNs-PSO analytics microservice platform | Combines prediction, optimisation, and smart-home control. | Control optimisation is outside the present tracker scope; no control-performance comparison is made. |
| [15], 2024 | Real-time energy-meter monitoring | Arduino/GSM smart-meter prototype | Reports a physical meter with monitoring and alert functions. | Provides physical-meter context; the present work does not claim meter calibration or SMS-alert validation. |
| [17], 2025 | Smart-city energy management | Distributed microservices, Kafka, and InfluxDB | Uses service and storage technologies relevant to energy data processing. | Technology overlap supports design rationale; city-scale results do not establish local residential performance. |
| [19], 2026 | Appliance monitoring and anomaly detection | Fast DDS and multi-task neural inference | Reports a real-time sensing-to-inference pipeline with a different middleware and model. | Illustrates a more extensively evaluated pipeline; the present study does not claim comparable appliance-level or field results. |
| [20], 2025 | Residential consumption and power-quality monitoring | IoT smart meter with power-quality analysis | Combines consumption monitoring with disturbance analysis in a physical prototype. | Power-quality classification and physical validation are outside the demonstrated scope of the present study. |
| [5]–[7], 2016–2017 | Household energy-data provenance | REFIT descriptor and cleaned-data records | Documents recorded household aggregate and appliance measurements and the cleaned-data distribution. | Dataset/provenance resource, not an anomaly-model benchmark; supports source verification and preprocessing rationale. |

## 2.6 Research Gaps Identified

### Research Gap 1: Traceability across the monitoring pipeline

The literature supports separation of monitoring, storage, analytics, and control concerns, but the evidence required to diagnose a single record remains less explicit in many architectural descriptions. A failure can occur before admission, during broker delivery, at persistence, during model handling, or when an alert policy applies. Without a declared boundary for each stage, it is difficult to distinguish a rejected request from a lost, duplicated, or policy-suppressed event.

The present project addresses part of this problem by exposing a gateway, ingestion service, Kafka topic, usage consumer, time-series store, model service, and alert service as separate components. Its source also includes code-level retry and dead-letter handling. However, the present study does not establish complete correlation-based reconciliation across those services, exactly-once semantics, or multi-instance alert correctness. The gap is therefore

addressed at the level of inspectable architecture and selected local evidence, not closed by a broad reliability claim.

Prior work establishes modular cloud and microservice designs for energy data, including real-time monitoring deployments [1]–[3], [17], [19]. However, the reviewed literature does not always report a record-level, inspectable connection from telemetry admission through broker delivery, persistence, inference, and alert decision in the setting of a local residential prototype. This matters because a missing or duplicated alert cannot otherwise be localised to admission, delivery, storage, inference, or policy handling. The present architecture exposes these boundaries through the gateway, Kafka, services, InfluxDB, and MySQL; end-to-end reconciliation, duplicate suppression, delivery, and fault recovery under recorded-data replay remain unverified.

**Research Gap 2: Separation of reproducibility from model validity**

The reviewed anomaly studies confirm that model metrics are protocol-specific. A repeated training run, a thresholded response, and a hold-out score are not equivalent measures. They answer, respectively, whether a procedure can be recreated, whether a service can execute, and how a fixed model behaved on a particular evaluation slice. Confusing these evidence types is especially risky when a historical CSV lacks transparent label-generation and feature-provenance documentation.

This project resolves the reporting problem by separating the legacy supervised result, its exact offline reconstruction, the Wokwi-compatible interface check, and REFIT preprocessing. The separation improves evidence clarity; it does not make the legacy model more accurate or create an external validation set. Objective 3 therefore remains an objective about reproducibility and bounded interpretation rather than a claim of generalisation.

Smart-meter studies report outcomes under their own datasets and protocols [4], [8], [10], [12], [19]. What is not interchangeable is the meaning of a historical metric, an interface response, and a source-integrity check. This study separates the historical Random Forest evaluation, the Wokwi-compatible smoke test, and REFIT preprocessing so that a repeated training run, endpoint response, or checksum is not misrepresented as household-level anomaly accuracy. The remaining limitation is that this separation improves interpretation, not external model validity.

**Research Gap 3: Reproducible recorded-household preparation and later runtime evaluation**

A household-data experiment must establish more than a file download. The experiment needs a source-verifiable cohort, a stated timestamp policy, a physically coherent transformation from power to energy when the target contract expects energy, an identity-to-device mapping, and a label protocol that belongs to the evaluated signal. These

requirements are coupled: a correct interval calculation does not prove that the payload is authorised, and an authorised payload does not turn appliance-level annotations into aggregate household labels.

The current study addresses the preparation part of this gap through checksum-verified REFIT inputs, a H1/H2/H4 primary cohort, a separate H3/H11 solar-sensitivity cohort, actual-delta interval estimates, and byte-identical outputs. The remaining runtime and semantic stages are deliberately open. Any future claim about REFIT accuracy must first show a verified derived_interval_energy_kwh-to-energyConsumed contract, isolated devices and credentials, request/event/persistence reconciliation, and a defensible aggregate-label alignment procedure.

REFIT establishes documented household measurements and cleaned-data records [5]–[7], but a reproducible recorded-household experiment also requires explicit cohort selection, timestamp handling, power-to-energy transformation, source integrity, device provisioning, and label alignment. These requirements matter because file integrity alone does not show that authenticated ingestion succeeds, that events reconcile through the runtime, or that an anomaly label belongs to the aggregate reading being evaluated. The present study contributes checksum-verified inputs, a defined H1/H2/H4 primary cohort, a separate solar-affected H3/H11 sensitivity cohort, and deterministic interval-energy artefacts. Device provisioning, live replay, aggregate-label validation, and REFIT accuracy remain outside the current evidence.

**Synthesis of Research Gaps**

Together, the three gaps map directly onto the four objectives. Objectives 1 and 2 establish inspectable admission and processing boundaries; Objective 3 provides a reproducible historical baseline while preserving its external-validity limitation; and Objective 4 provides a controlled recorded-household input for the later validation sequence. The connection is methodological: it specifies what evidence would be needed at the next stage rather than claiming that all stages have already been completed.

The three gaps are connected: inspectable software boundaries support traceability; evidence separation prevents an architecture or smoke test from being mistaken for model validation; and reproducible REFIT preparation supplies a controlled basis for a later recorded-household experiment. Together they motivate the four objectives: implement the pipeline, provide observability, reproduce the legacy baseline without overstating it, and prepare REFIT data for future validation.

**Problem Statement**

The research problem is to document and evaluate a residential energy-monitoring system in a way that preserves the distinction between implemented software, reproduced historical results, data-integrity evidence, and unperformed runtime experiments. The

system must support simulator-compatible telemetry, event processing, storage, analytics, inference, and alert handling through inspectable boundaries, while REFIT preparation must remain reproducible and ready for a later controlled end-to-end evaluation.

## 3. Materials and Methods

### 3.1 Overall System Methodology

#### 3.1.1 Evidence Streams and Study Boundary

The methodology uses deliberately separated evidence streams. The first stream is implementation evidence drawn from service code, configuration, schemas, and architecture documentation. The second is dated local runtime evidence from controlled requests, authorization checks, storage observations, alert cases, and single-host tests. The third is an offline reconstruction of the supplied legacy supervised experiment. The fourth is checksum and preprocessing evidence for the REFIT cohort. The results section keeps these streams distinct because their inputs, controlled conditions, and permissible conclusions differ.

This separation prevents a common interpretation error. An implemented component can be inspected statically without being exercised under a workload; a dated runtime check can establish a specific observed path without establishing its current post-hardening behaviour; and a deterministic preprocessing run can prove that an artifact is reproducible without proving that the application accepted it. The methodology therefore records both the positive evidence and the boundary that remains open.

The system accepts a Wokwi-compatible telemetry payload containing deviceId, energyConsumed, and timestamp. User-facing requests are governed by JWT, role, and ownership checks; device telemetry uses a device-bound token. The Ingestion Service validates the telemetry contract before publishing energy-usage events to Kafka.

The minimal telemetry contract is intentionally small: deviceId, non-negative energyConsumed, and timestamp. The contract does not contain an event identifier or an explicit unit field. Consequently, the ingestion layer can reject a structurally invalid reading, but it cannot infer whether a client has supplied watts, kWh, or a double-converted value merely from the JSON schema. This is a decisive limitation for REFIT: the derived interval-energy field is a documented future compatibility candidate, not an accepted live mapping.

For a user-originated call, the security design uses a JWT-derived principal, role checks, and ownership context. For telemetry that represents a device client, a device-bound token is presented to the gateway. The gateway removes externally supplied privileged headers before a valid machine-token request is forwarded. At the ingestion layer, authorization is checked again: a user path is verified against device ownership, while a machine path is verified against the exact device identifier in the payload. This defence-in-depth

arrangement is implemented locally, but it is not a demonstration of mTLS, automatic credential rotation, or independent security assurance.

The Usage Service consumes energy-usage events and records time-stamped usage together with defined-period aggregates. InfluxDB stores time-series usage, whereas MySQL stores household, device, and alert state. The baseline inference service consumes the appropriate event type and returns anomaly results. The Alert Service implements threshold and anomaly pathways with a configured 15-minute deduplication window. Prometheus and Grafana provide service observation.

After authorization, Ingestion constructs an EnergyUsageEvent and publishes it to the energy-usage topic. Usage consumes this event as part of the usage-service consumer group and writes a time-series point with deviceId as a tag, energyConsumed as a field, and the event timestamp as the write time. A scheduled task aggregates a preceding one-hour window every ten seconds, resolves household context, compares the sum with configured thresholds, and can publish an energy-alerts event. This describes the implemented route and its intended aggregation semantics; it should not be read as a metering or billing algorithm.

The usage consumer has a code-level failure policy for time-series persistence. A failed handler invocation receives the initial attempt plus two configurable retries, followed by publication to an energy-usage.DLT record if recovery is required. This is a bounded recovery design, not a completed recovery experiment. A future fault study would need to verify the retry count, dead-letter publication, partition behaviour, operator visibility, and absence of unintended duplicate accounting in the live configuration.

Architecture implementation, historical-baseline reproduction, live interface testing, and REFIT preprocessing are treated as separate evidence streams. This prevents a successful service response or one inference result from being counted as an accuracy, reliability, or alert-correctness result.

The architecture evidence is complemented by dated local observations rather than replaced by them. For example, a 2026-10-11 check recorded unauthorized telemetry rejection, device-token mismatch rejection, accepted owner and device submissions, and the later appearance of 0.37 kWh and 0.19 kWh as 0.56 kWh in the Usage API. These observations support a particular local Gateway-to-Ingestion-to-Kafka-to-Usage-to-InfluxDB route. They do not prove long-duration ordering, sustained consumer behavior, universal fault recovery, or the REFIT path.

A separate 2026-10-05 single-host run provides historical evidence for controlled alert and load scenarios. It predates later private-caller and device-token hardening, so it is reported only as historical local evidence and not as proof of the final hardened route. Treating the chronology explicitly prevents older runtime observations and later source-level checks from being pooled into a stronger unsupported claim.

### 3.2 Proposed System Architecture

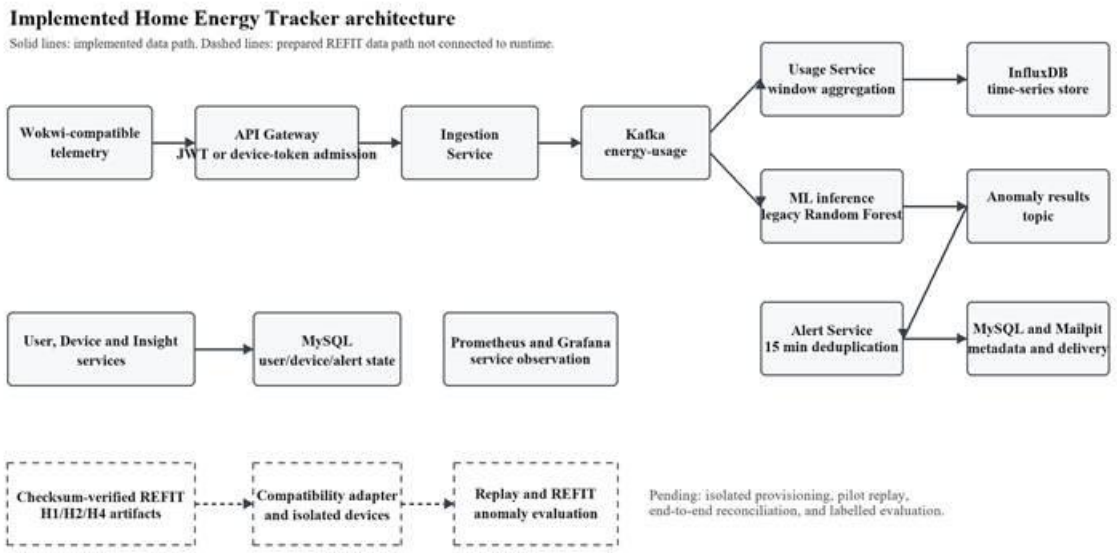



Figure 1 presents the implemented Home Energy Tracker architecture and the separately prepared, but not runtime-connected, REFIT branch.

The solid path runs from simulator-compatible telemetry through gateway admission, ingestion, Kafka, usage aggregation, time-series storage, baseline inference, alert processing, relational state, and observability. The dashed REFIT path represents preparation for isolated provisioning, replay, reconciliation, and labelled evaluation; those stages have not yet been completed.

At the public boundary, the frontend interacts with the API Gateway, which routes to the User, Device, Ingestion, Usage, Insight, and Alert services. Keycloak provides the OpenID Connect and JWT identity context for interactive use. The explicit gateway role is significant: service routes are not intended to decide public authentication independently of the boundary context, and internal headers used for trusted downstream calls are not meant to be supplied by an external browser or device.

The data plane begins at Ingestion, where a validated request becomes an energy-usage event. Kafka isolates that publication from persistence and subsequent consumers. Usage owns time-series writes and rolling sums; InfluxDB stores the resulting time-stamped usage values. MySQL holds the relational facts needed for profiles, devices, and alerts. Alert Service consumes threshold and anomaly events, persists a non-suppressed alert, and may attempt e-mail delivery. Mailpit is a local delivery sink used in dated validation records rather than evidence of a production e-mail service.

The baseline FastAPI service loads a persisted model and can expose prediction information, including imputed feature names when optional contextual fields are absent. Prometheus and Grafana are configured for service observation. Their presence supports inspectability of

selected operational signals, but configured dashboards and metrics are not equivalent to a completed observability or incident-response evaluation. The figure therefore distinguishes the implemented solid path from the dashed REFIT branch, where provisioning, replay, reconciliation, and household-level evaluation remain pending.

The architectural rationale is separation of responsibilities, not a claim that every service must run independently in a production deployment. The local Docker composition contains multiple supporting services and is resource constrained. Its deployment analysis has shown that the main stack and a future isolated REFIT provisioning stack should not run concurrently on the documented 8 GB workstation.

**Table II. System components and evidence boundaries**

| Component | Implemented behaviour | Evidence boundary |
|---|---|---|
| Admission | Keycloak-backed user routes and device-token telemetry admission at the gateway. | Functional architecture and some authorization controls; no production security evaluation. |
| Brokering and ingestion | Ingestion validates telemetry and publishes Kafka energy-usage events. | Historical/local event-path evidence; no REFIT replay has been run. |
| Storage and usage | Usage aggregation writes time-stamped usage to InfluxDB; household, device, and alert state are retained in MySQL. | Component and historical integration evidence; no utility-bill verification. |
| Baseline inference | A FastAPI service loads the fixed Random Forest and returns anomaly results. | REFIT accuracy is not established by historical-dataset evaluation or a live smoke test. |
| Alerting and monitoring | Threshold/anomaly paths reach Alert Service; a 15 min deduplication window and Prometheus/Grafana observation are configured. | Historical local evidence exists; fault and replay tests after isolation remain pending. |
| REFIT data path | Checksum-verified, normalised H1/H2/H4 artefacts are prepared separately. | Device provisioning, ingestion, alert assessment, and tagged-anomaly evaluation are unfinished. |

### 3.3 Identity, Admission, and Event Processing

The gateway defines the API boundary: user-facing routes use JWTs with role and ownership checks, whereas simulator-compatible device telemetry uses a device-bound token. A REFIT household identifier is not assumed to equal a backend device identifier. Any future replay must provision households, devices, and credentials separately.

The user-facing and device-facing security paths have different inputs and decisions. A user request has an authenticated principal and an ownership rule: a USER may operate only on the household context carried through the trusted gateway boundary, while an ADMIN path is separately authorised. A device request carries a device token in the designated header

and is checked against the payload deviceId. The configuration-based mapping is deliberately not equated with a database-generated token lifecycle; device creation persists device metadata and ownership, but does not by itself generate, rotate, or store a device credential.

The gateway treats caller-supplied internal headers as untrusted. For machine-token traffic, it strips public attempts to set privileged user or gateway-proof headers before forwarding the request. For interactive traffic, the trusted downstream context is derived from the authenticated principal. Ingestion then fails closed if its ownership lookup or token mapping check does not authorize the input. Constant-time comparison is used for token matching. These implementation details make the boundary more explicit, but they do not establish resistance to every threat model or prove production secret-management practice.

The minimum DTO requires a non-null device identifier, a non-negative energy value, and a timestamp. The validation prevents missing or negative contract values from proceeding through the normal route. It does not validate semantic plausibility, expected household power, or source calibration. In particular, a device identifier cannot be assigned from a REFIT house number merely because the two numbers happen to match; any replay requires separately provisioned experiment profiles, devices, and unique credentials.

After contract validation, energy-usage events are published to Kafka for downstream aggregation and baseline inference. Decoupling enables component-level inspection, but component availability or Kafka connectivity does not by itself prove end-to-end event reconciliation.

The EnergyUsageEvent carries the device identifier, energyConsumed value, and timestamp. It is not an enriched analytical record and does not contain a durable correlation identifier for every downstream boundary. This fact limits the precision of any current claim about per-record auditability. Future replay work must create an external request log and reconciliation ledger rather than assume that ordinary broker offsets or database rows alone solve end-to-end traceability.

Source-level tests exercise several expected security and processing behaviours, including rejection before Kafka publication when authorization fails, trusted-header handling, and selected usage-access rules. These tests strengthen confidence in component logic but do not replace a live Keycloak, gateway, broker, database, and e-mail integration study. The paper accordingly labels them as component evidence rather than as a full system-security result.

### 3.4 Baseline Model and Alerting Boundary

The retained Random Forest artefact, random_forest-csv-20261003T205417Z, uses Electricity_Consumed, Temperature, Humidity, Wind_Speed, Avg_Past_Consumption, Hour, DayOfWeek, and ConsumptionDeviation. The historical protocol uses a chronological split of

3,500 training, 750 validation, and 750 test records; the decision threshold is selected from validation F1.

The legacy training procedure begins with five supplied row-level predictors: Electricity_Consumed, Temperature, Humidity, Wind_Speed, and Avg_Past_Consumption. It derives Hour and DayOfWeek from the timestamp and computes ConsumptionDeviation as Electricity_Consumed minus Avg_Past_Consumption. Records are parsed, invalid numeric fields are removed, negative electricity values and complete duplicates are excluded, and the remaining rows are stably ordered chronologically. The procedure does not construct lagged or rolling features from later observations, but the causal provenance of the supplied Avg_Past_Consumption field remains undocumented.

The data are divided into 3,500 training rows, 750 validation rows, and 750 held-out test rows. Validation F1 selects the decision threshold over the documented candidate range, yielding 0.31 for the retained Random Forest. The held-out test is not used to select that threshold. The wider trainer contains a deterministic test PR-AUC tie-breaker in its model-selection logic, so the paper avoids the broader unsupported statement that test data played no role in every model-selection decision.

At inference time, a Wokwi-compatible payload may omit temperature, humidity, wind speed, and Avg_Past_Consumption. The service fills absent contextual values with training medians, derives temporal features from the supplied timestamp, obtains a probability from the fixed artifact, and applies the stored threshold. It reports the imputed feature names so that a three-field payload is not silently mistaken for a fully observed feature vector. This interface decision is transparent, but median imputation is not an evaluation-quality substitute for a real REFIT feature contract.

The model uses 300 trees, min_samples_leaf = 2, class_weight = balanced_subsample, random_state = 42, and a 0.31 threshold. For Wokwi-compatible payloads, four contextual inputs are median-imputed and temporal features are derived from the timestamp. The resulting response is an interface smoke test, not a statistical evaluation of the historical model.

The retained model configuration is 300 trees, min_samples_leaf = 2, class_weight = balanced_subsample, and random_state = 42. The saved training artifact can use broad parallelism while the deployed inference process limits the effective worker count to reduce oversubscription on the local host. This operational setting is a resource-management decision, not an optimisation result. The paper reports it only to explain why a local in-process benchmark cannot be extrapolated to the total request path.

The Alert Service has a configured 15-minute deduplication window for threshold and anomaly pathways. No replay or fault experiment has yet established duplicate suppression, notification delivery, latency, recovery behaviour, or fault tolerance.

Alert Service receives both threshold-generated energy-alerts and ML-generated anomaly-results. A normal model decision creates no anomaly alert. For an alert candidate, the service forms a key from the user context, a device or household scope, and alert type; it searches for a recent matching alert and suppresses a duplicate inside the configured 15-minute window. A non-suppressed alert is persisted, and the e-mail attempt/sent result is recorded separately. Thus, persistence and e-mail delivery are not conflated in the implementation.

A focused mock-based test on 2026-10-10 executed first-alert, in-window duplicate, post-window, and user/device separation cases with four passing tests. A dated 2026-10-05 local duplicate scenario also recorded two accepted synthetic submissions, one stored ML_ANOMALY alert, and a suppression log. These are useful bounded checks, but they do not prove cluster-safe atomic deduplication, arbitrary replay behaviour, full database failure recovery, or REFIT-specific notification correctness.

**3.5 REFIT Preprocessing & Interval-Energy Estimation**

The five local cleaned REFIT files (H1, H2, H3, H4, and H11) were checked against the official Zenodo MD5 values. Aggregate total-house active power is expressed in watts. H1, H2, and H4 form the primary cohort, while H3 and H11 remain a solar-affected sensitivity cohort because no solar or net-export correction was applied. Source-file MD5 verification is distinct from the SHA-256 hash recorded for the derived primary output.

REFIT is used as a recorded-household source for a future experiment, not as a direct replacement for the legacy labelled CSV. The five local cleaned files were checked against official Zenodo MD5 values before their headers and quality characteristics were inspected. The primary cohort consists of Houses 1, 2, and 4 because each has a selected 14-day window satisfying the pre-registered quality gates and no documented solar-interference caveat applied to its primary record. Houses 3 and 11 are retained as a solar-sensitivity cohort because the source documents solar-generation interference and no net-export correction has been performed.

The cleaned files contain a human-readable Time field, a numeric Unix timestamp, Aggregate, appliance channels, and Issues context. Unix is used to establish chronological order because the human-readable timezone convention has not been independently verified for this analysis; the Time string is retained verbatim. Aggregate is treated as whole-house active power in watts. It is not silently relabelled as energy, and the Issues column is not converted into an anomaly label.

Each selected pair of adjacent records yields a candidate interval. The method uses the aggregate power at the start of the interval and the observed elapsed duration, so it does not blindly multiply every row by the nominal eight-second cadence. Only intervals with $0 < \Delta t \leq 16$ s and a non-negative start power are included. A longer cross-gap interval is

excluded rather than filled, capped, interpolated, or synthesised. This policy avoids hiding the energy contribution of an unobserved period behind an unsupported assumption.

For each eligible pair of consecutive records, interval energy is estimated from the first aggregate-power sample and the actual elapsed time. The following formula preserves the documented method and units.

In dimensional terms, watts multiplied by seconds produces joules. Dividing by 3,600,000 converts watt-seconds to kilowatt-hours. The left-rectangle/sample-hold interpretation assumes the first aggregate sample represents the following observed interval; it is a documented numerical estimate, not proof that REFIT readings are point values rather than interval averages. This semantic limitation is preserved for the later compatibility review.

$$E_{\mathrm{kWh}} = \frac{P_{\mathrm{start,W}} \times \Delta t_{\mathrm{s}}}{3{,}600{,}000}$$

Only intervals satisfying the condition below are retained.

The maximum 16-second rule permits a modest tolerance around the nominal cadence while preventing the integration of an unobserved long gap as though it were a continuous measurement. For each excluded candidate, the source record and reason remain in the audit output. At the end of a selected window, the final record has no successor and is likewise retained in the source audit rather than contributing an invented interval.

$$0 < \Delta t_{\mathrm{s}} \leq 16\ \mathrm{s}$$

Cross-gap intervals are excluded rather than interpolated, filled, capped, or synthesised. Negative-power records are excluded from the combined interval-energy output but retained in the source audit. IQR flags are diagnostic only, and REFIT Issues values are treated as contextual information rather than anomaly labels.

The generated artifacts include cleaned primary and sensitivity cohorts, a manifest, per-household quality report, timestamp-interval report, excluded-record log, and preprocessing summary. IQR fences flag unusual values diagnostically but do not remove a valid aggregate value. This distinction matters: an unusual power level may be an appliance or household event, a sensor condition, or simply a legitimate observation. It cannot be called an anomaly label without a verified task-specific annotation policy.

The primary output contains 468,847 retained intervals across the three windows. Its SHA-256 hash is recorded separately from the five source-file MD5 values. Repeating the preprocessing against the same checksum-verified files and configuration produced byte-identical output artifacts. Source integrity, derived-artifact identity, and live system acceptance are therefore three different claims, and only the first two are completed here.

**3.6 Algorithmic Procedure for REFIT Normalisation**

**Algorithm 1. REFIT interval-energy normalisation**

- Input: a cleaned, verified REFIT household file for a selected 14-day period, checked against its MD5 checksum.
- Sort records by preserved Unix timestamp and retain the associated Time value.
- Compute the actual elapsed interval Δt for every adjacent pair.
- If $0 < \Delta t_s \leq 16$ s and the first aggregate-power sample is non-negative, apply the left-rectangle/sample-hold rule to estimate interval energy.
- Otherwise, retain the source record for audit and record the interval in the excluded-record log; do not interpolate, cap, or synthesise values.
- Produce the cleaned cohort, preprocessing summary, quality report, timestamp-interval report, and excluded-record log.
- Create the H1/H2/H4 primary cohort and retain H3/H11 as a separate solar-affected sensitivity cohort.

**Algorithm 2. Telemetry Admission and Event Publication**

Input: an HTTP telemetry request containing deviceId, energyConsumed, timestamp, and either an authenticated user context or a device-bound token. Output: a rejected request or an authorised energy-usage event published to Kafka.

1. Reject a request with missing required fields or a negative energyConsumed value. 2. If a machine token is presented, remove untrusted caller-supplied privileged headers and verify that the token matches the payload deviceId. 3. Otherwise, require trusted gateway context and verify that the authenticated user owns the payload device. 4. Reject the request if either authorization path fails or if the required ownership lookup cannot be completed. 5. Construct an EnergyUsageEvent with deviceId, energyConsumed, and timestamp. 6. Publish the event to the energy-usage topic. 7. Return the ingestion result without claiming that downstream persistence, inference, or alerting has already completed.

This algorithm expresses the source-level service contract. It is not a replay algorithm and does not introduce an idempotency key, unit conversion, or a cross-service correlation identifier that the current contract does not contain.

**3.8 Evidence and Reproducibility Workflow**

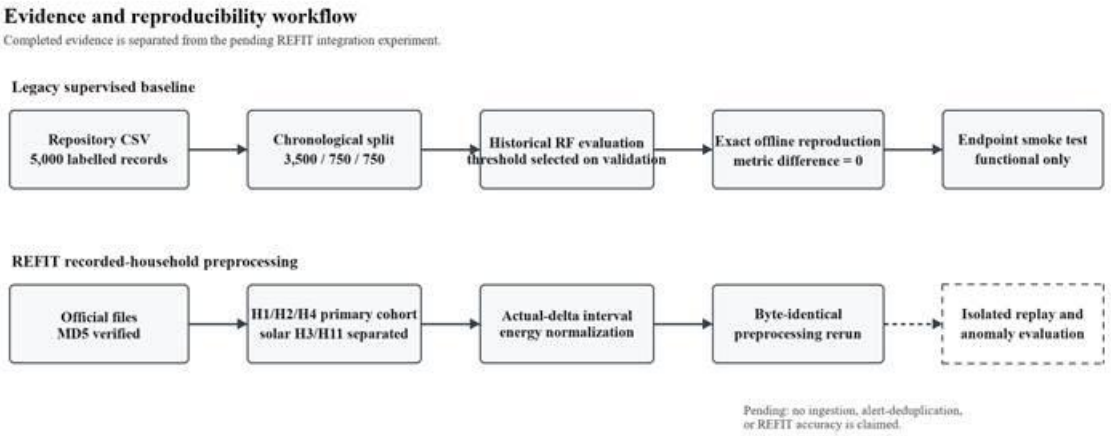



Figure 2. Evidence and reproducibility workflow. The REFIT runtime phase remains pending until isolated device provisioning, credential creation, pilot reconciliation, and verified evaluation are completed.

The upper workflow records the historical 5,000-record protocol, validation-threshold selection, offline reproduction, and endpoint smoke-test boundary. The lower workflow records official REFIT verification, cohort separation, actual-delta normalisation, and byte-identical repeatability. The final stage is deliberately dashed because ingestion and anomaly evaluation have not yet occurred.

### 3.9 Reproducibility Protocol

The historical baseline protocol records a copied 5,000-row input, the stable chronological split, fixed preprocessing, model configuration, validation threshold rule, and artifact-hash comparison. A rerun reproduces the reported metrics exactly and matches the stored Random Forest artifact hash. This establishes a repeatable local procedure under the available environment. It does not verify the external source, physical units, labels, or causal availability of all predictor values.

The REFIT protocol records a different reproducibility chain: official local-file MD5 verification, declared source columns, a cohort-selection file, a preprocessing configuration, an excluded-record log, quality reports, derived SHA-256 values, and an independent rerun. The byte-identical result demonstrates deterministic transformation of the selected data. It does not demonstrate a Kafka replay, database write, live alert, or anomaly evaluation.

For this reason, the workflow figure should be read as an evidence map rather than a claim that every arrow has been executed with REFIT. The solid and completed branches have different evidence sources from the dashed future branch. This presentation permits the next experiment to be audited against explicit inputs and acceptance checks.

## 4. Experimental Results and Discussion

### 4.1 Experimental Conditions and Evidence Classes

The reported observations come from distinct evidence classes. The architecture identifies implemented services and event paths; the historical analysis concerns the supplied labelled CSV; offline reproduction checks whether the historical experiment can be repeated; and REFIT preprocessing checks source integrity and deterministic preparation. None of these observations alone is a new anomaly-accuracy result. In particular, a service response or repeated training run does not establish accurate operation on household data.

The evidence was collected at different times and with different scopes. Dated local Docker checks on 2026-10-05 covered controlled synthetic integrations, a single-host load scenario, alert cases, and selected recovery observations. Later source hardening and component tests were recorded on 2026-10-10 and 2026-10-11. Offline supervised reconstruction and input-contract checks were recorded on 2026-10-15. REFIT preprocessing was performed separately. The study does not combine these dates into a single homogeneous benchmark because the deployed configuration and tested boundary differed.

This chronology changes the interpretation of a pass. For example, a pre-hardening local load result gives bounded evidence about the older local endpoint, whereas later component tests show expected filter and authorization behaviour without proving a live Docker route. Both are useful, but neither can be relabelled as a current production-capacity or end-to-end security result.

The Windows laptop used for local work has 8 GB RAM and a Docker memory limit of 3.696 GiB; the observed main-stack requirement is approximately 2.89 GiB. The isolated REFIT provisioning stack should therefore not run concurrently with the main stack on this machine. This resource constraint is a deployment limitation, not evidence of scientific validation.

The published environment evidence gives an 8 GB Windows laptop, a Docker memory limit of 3.696 GiB, and an observed main-stack requirement of approximately 2.89 GiB. These values are deployment constraints, not system-performance metrics. They motivate strict separation of the main stack and any future isolated REFIT provisioning stack, since concurrent operation could destabilise the demonstration environment or contaminate experiment-only state.

### 4.2 Software Components and Data Sources

The local composition includes an API gateway, Keycloak, device-token admission, Kafka, InfluxDB, MySQL, a FastAPI Random Forest inference service, the Alert Service, Prometheus, and Grafana. Version numbers not established by the available evidence are not assumed. The legacy supervised CSV is repository material, whereas REFIT is an official recorded-household source with separate dataset records [5]–[7].

The components are grouped by responsibility rather than treated as interchangeable infrastructure. The gateway and Keycloak form the public identity and routing boundary. User and Device services hold household/device context; Ingestion accepts authorised telemetry; Kafka transports energy-usage events; Usage writes and aggregates time-series values; InfluxDB stores those values; the ML service exposes the retained inference contract; Alert Service persists and sends alert outcomes; Insight Service supplies deterministic user-facing calculations; and Prometheus/Grafana provide monitoring. MySQL supports transactional state including users, devices, and alerts.

A dated runtime validation found the gateway health endpoint, Keycloak discovery, frontend, Prometheus readiness, and Grafana login route reachable in the local composition. It also confirmed selected authorization outcomes and the 0.56 kWh controlled usage result. An optional ML container was intentionally not included in that particular 2026-10-11 validation because the model directory was inaccessible to Docker on the workstation. The paper therefore treats ML endpoint evidence separately from broader service availability.

### 4.3 Historical Baseline Protocol

The historical protocol evaluates the original Random Forest with a chronological 70/15/15 split: 3,500 training, 750 validation, and 750 test records. The threshold is selected using validation F1. The logistic-regression comparator uses the same held-out split with a threshold of 0.28.

The copied legacy CSV contains 5,000 records with 4,750 Normal and 250 Abnormal labels. The chronological partitions contain both classes: 3,335 Normal and 165 Abnormal rows in training, 705 and 45 in validation, and 710 and 40 in testing. The reconstruction also found zero timestamp, complete raw-record, and engineered-feature fingerprint overlap across the partitions. These checks reduce obvious row leakage under the stored protocol, but they do not resolve how Avg_Past_Consumption was constructed before the CSV entered the repository.

The logistic-regression comparator used the same held-out split and its own validation-selected threshold of 0.28. Table III is thus an internal comparison under one supplied dataset rather than a contest against the external methods in Section 2. The Random Forest's higher F1, ROC-AUC, and PR-AUC are meaningful only within this legacy protocol and should not be interpreted as an estimate of utility fraud detection, appliance anomaly detection, or performance on recorded REFIT households.

### 4.4 Offline Reproduction and Live Smoke Test

The deterministic trainer was run on the 5,000 copied historical records. All reported metric differences were zero, and the SHA-256 hash of the reproduced Random Forest artefact matched the legacy artefact, showing that the locally available historical experiment was reproduced. A separate Wokwi-compatible request used deviceId 1, energyConsumed =

0.55, and timestamp 2026-10-11T00:00:00Z. Its response tests model loading and payload handling only; it is not a latency, alert-delivery, deduplication, recovery, or accuracy test.

The offline reconstruction verifies more than a single displayed score. All reported metric differences between the stored and rerun comparison files were zero, and the reproduced Random Forest artifact hash matched the legacy artifact. On the fixed copied held-out CSV, a stratified bootstrap calculation also yielded a conditional 95% F1 interval of 0.838300 to 0.957444. This interval describes resampling variability of that particular held-out dataset and fixed model; it is not a population confidence interval for unseen households.

Five in-process validation cases passed for the ML input contract. A separate six-case synthetic Wokwi-scale check found that five values lay above the historical Electricity_Consumed maximum of 1. These results show that syntactically acceptable interface values can be outside the observed historical range. They do not tell whether the anomaly decision is correct for those values and motivate a future out-of-distribution or contract-range policy.

For the actual three-field smoke request, deviceId 1, energyConsumed 0.55, and timestamp 2026-10-11T00:00:00Z produced anomaly = false, probability = 0.0, threshold = 0.31, and four explicit imputed contextual fields. The observation establishes that the fixed model loaded and processed the input according to the service's imputation rule. It does not measure the distribution of response times, broker-to-alert operation, or classifier accuracy.

**4.5 REFIT Data Configuration and Repeatability**

The H1, H2, and H4 primary-cohort windows are reported in Table IV. The preprocessing pipeline was run twice with the same five validated source files, criteria, and configuration. Both artefact sets were byte-identical and included the cleaned primary and solar-sensitivity cohorts, manifest, excluded-record log, quality report, preprocessing summary, and timestamp-interval report.

The primary-cohort summaries must be read as interval counts rather than independent household examples. H1 contains 153,578 retained intervals and an estimated 104.820956111 kWh after 184 excluded candidates; H2 contains 154,985 retained intervals and an estimated 170.354373889 kWh after 112 exclusions; and H4 contains 160,284 retained intervals and an estimated 111.223081389 kWh after 40 exclusions. The excluded counts are dominated by cross-gap candidates plus the terminal record without a successor, not by a model decision.

The selected H1, H2, and H4 windows passed the stated reconciliation and timestamp-quality checks. Native-cadence shares were 95.97%, 95.64%, and 99.30%, respectively. Their IQR diagnostics flag many unusual aggregate values but do not remove them; this preserves the distinction between a numerical distributional flag and a verified anomaly label. H3 and

H11 remain outside the primary summary because documented solar-generation interference has not been corrected to a consumption-only or net-import signal.

The byte-identical rerun and primary SHA-256 establish that another execution using the same verified inputs and configuration reproduced the same derived files. This is strong evidence for preprocessing repeatability, not a claim that the interval count represents 468,847 independent evaluations or that the estimated 386.398411389 kWh has passed the live energyConsumed contract.

**Table III. Historical baseline, offline reproduction, and live smoke-test evidence**

| Evidence item | Protocol | Observed result | Permissible interpretation |
|---|---|---|---|
| Historical Random Forest | Chronological 70/15/15 split; 750 test records; validation-tuned threshold. | Precision 0.880952; Recall 0.925000; F1 0.902439; ROC-AUC 0.997430; PR-AUC 0.948532. | Historical CSV-specific evaluation, not REFIT or a new field result. |
| Historical logistic regression | Same held-out split; threshold = 0.28. | Precision 0.053333; Recall 1.000000; F1 0.101266; ROC-AUC 0.422852; PR-AUC 0.059710. | Legacy comparator only. |
| Offline reproduction | Current deterministic trainer with 5,000 copied records. | All reported metric differences = 0; reproduced Random Forest artefact SHA-256 matched the legacy artefact. | Reproduces the legacy experiment; does not prove external validity. |
| Live inference smoke test | deviceId 1; energyConsumed 0.55; timestamp 2026-10-11T00:00:00Z. | anomaly = false; probability = 0.0; threshold = 0.31; four context features imputed. | Confirms endpoint execution and declared imputation only. |

**Table IV. REFIT primary-cohort normalisation results**

| Home | Time window (UTC) | Chosen rows | Intervals retained | Rows excluded | Energy estimated (kWh) | Quality output |
|---|---|---|---|---|---|---|
| H1 | 2014-10-01 to 2014-10-15 | 153,762 | 153,578 | 184 | 104.820956111 | PASS; 95.97% native cadence |
| H2 | 2014-10-01 through 2014-10-15 | 155,097 | 154,985 | 112 | 170.354373889 | PASS; 95.64% native cadence |

| | | | | | | |
|---|---|---|---|---|---|---|
| H4 | 22 Apr 2015 to 6 May 2015 | 160,324 | 160,284 | 40 | 111.223081389 | PASS; 99.30% native cadence |
| Total primary | Three 14-day periods | 469,183 | 468,847 | 336 | 386.398411389 | All record reconciliations passed |

Here, aggregate denotes total household active power in watts. Interval energy is estimated by the actual-delta, left-rectangle, sample-hold method. H3 and H11 are excluded from the primary summary because of the documented solar-interference sensitivity.

**4.6 Results and Discussion**

Table III shows that the Random Forest exceeds the logistic-regression comparator on historical held-out F1, ROC-AUC, and PR-AUC. Exact offline reproduction and a matching artefact hash establish that the legacy experiment can be regenerated from the locally available copied input and configuration. They do not resolve the historical CSV's missing provenance, licence, label-generation procedure, household identity, or the causal construction of Avg_Past_Consumption.

The historical comparison supports a narrow conclusion: under the available CSV, feature preparation, chronological split, and threshold protocol, the retained Random Forest outperformed the retained logistic-regression comparator. It does not show that Random Forest is universally superior to regression, autoencoding, dynamic-thresholding, or other approaches reviewed in Section 2. The model choice is a legacy artifact result whose transparency is improved by exact reconstruction and explicit limitations.

The reproducibility result is still useful. An exact repeated score and model hash make the historical baseline inspectable and reduce ambiguity about configuration drift. They cannot repair missing documentation outside the repository. The unresolved source, licence, label-generation method, household identity, physical units, and Avg_Past_Consumption construction remain material threats to any broader scientific interpretation.

The smoke test shows that the deployed interface can load the fixed model and handle a Wokwi-compatible payload with declared imputation. It provides no response-time distribution, throughput, alert-delivery, deduplication, recovery, or statistical-accuracy evidence. Five of six examined simulator boundaries also fall outside the historical 0–1 Electricity_Consumed range, motivating future input-range and out-of-distribution controls.

Dated local runtime observations provide modest but genuine evidence beyond static code. The 2026-10-11 controlled inputs demonstrate selected authorization decisions and a Gateway-to-Usage persistence path. Earlier 2026-10-05 local evidence records a 450-request ingestion run with 450 accepted checks, a mean HTTP duration of 39.44 ms, and p95 of

109.77 ms under a ramping single-host scenario. These values are historical local endpoint observations, not an inference benchmark, broker throughput measure, or deployment-scale capacity result.

The dated local alert evidence is also specific. A controlled threshold case observed an alert with sent = true and a matching Mailpit message after 13.64 s, while a duplicate scenario recorded one stored alert and a suppression message inside the configured window. The later focused mock test covered four logical deduplication cases. Together they support the presence of a duplicate-aware policy and selected local behaviour, but they do not establish exactly-once handling, multi-instance atomicity, arbitrary replay correctness, or a REFIT-driven alert evaluation.

A controlled fault record observed ingestion acceptance while Mailpit was unavailable and later observed container-state recovery. A separate ML-container fault record did not establish post-restart Kafka reconnection or unique downstream event processing. The results are therefore reported as limited fault observations. They are not used to claim resilience of every service combination or end-to-end recovery of an alert lifecycle.

REFIT preprocessing produced a traceable primary cohort of 468,847 retained intervals and an estimated 386.398411389 kWh across three selected 14-day periods. The selected intervals contained no duplicate or out-of-order Unix timestamps, and each household satisfied the reported cadence, coverage, missing-slot, and maximum-gap checks. The two preprocessing outputs were byte-identical; the primary-output SHA-256 is dfdba435f35a1548661309fd67b267548f419edd489cacf25c7566098f189163.

The REFIT result makes a different contribution. It turns a documented historical household source into a deterministic, auditable interval-energy artifact while retaining the evidence needed to inspect exclusions and quality. The chosen H1/H2/H4 cohort provides a realistic basis for a future chronological stream, whereas H3/H11 make the solar-interference limitation visible instead of hiding it in a pooled total.

This result is a preparation and reproducibility result, not a model-effectiveness claim. REFIT appliance-level annotations are not treated as aggregate anomaly ground truth: only H1 overlaps the selected primary cohort, and annotation semantics, counts, timestamps, and alignment with aggregate readings remain unverified. The derived interval-energy mapping has also not been accepted by the live ingestion API.

No table in this paper should be read as a substitute for a pre-registered end-to-end study. Before a real REFIT evaluation, the project must verify that derived_interval_energy_kwh maps to the semantic unit expected by energyConsumed without a second conversion; create isolated H1/H2/H4 profiles and device-bound credentials; run a bounded pilot; reconcile submitted, accepted, persisted, and rejected events; and only then evaluate an aligned aggregate label protocol. Without those steps, a REFIT F1-score would not be defensible.

**Table V. Evidence boundaries and current validation status**

| Region | Proven outcome | Not configured or not validated |
|---|---|---|
| System implementation | Gateway, event processing, storage, deterministic insights, alert path, and observability are implemented in the local composition. | Production-scale capacity, full recovery behavior, and independent penetration security testing. |
| Baseline legacy | A dated endpoint smoke test passed, and offline reproduction of historical Random Forest metrics was exact with the copied CSV. | External data provenance, actual household generalisation, REFIT accuracy, and a complete post-restart ML-to-alert outcome. |
| REFIT data | Five local official files were MD5-validated; H1/H2/H4 interval artefacts were normalised and replicated byte-for-byte. | End-to-end ingestion, device/token provisioning, pilot/full replay, alert deduplication, and aggregate anomaly-label evaluation. |
| Deployment viability | Isolated-experiment design and safety assessments define necessary provisioning and storage boundaries. | Safe isolated deployment on the 8 GB laptop while the main stack is running; the measured memory constraint prevents concurrent operation. |

### 4.7 Objective-to-Evidence Interpretation

**Table VI. Objective-to-evidence status and remaining validation gates**

| Objective | Implemented or documented basis | Evidence in this study | Validation still required |
|---|---|---|---|
| O1 | Gateway admission, JWT/ownership checks, device-token verification, DTO validation, and ingress authorization. | Implemented source behaviour; dated local authorization checks. | Independent security assessment, credential lifecycle tests, and hardened live-route revalidation. |
| O2 | Kafka publication/consumption, InfluxDB persistence, MySQL state, configured retries/DLT, alert path, and observability components. | Dated local event-path, alert, and single-host load observations; source-level tests. | Correlation-based reconciliation, repeated performance studies, fault recovery, and multi-instance behaviour. |
| O3 | Fixed Random Forest, chronological split, validation thresholding, exact rerun, and model-artifact comparison. | Historical CSV metrics reproduced exactly; interface smoke test and in-process checks are separately scoped. | Documented external data source, causal feature provenance, and household-level evaluation. |
| O4 | Official REFIT checksum verification, cohort selection, actual-delta normalization, quality reports, and repeatability verification. | Byte-identical H1/H2/H4 preprocessing outputs and auditable interval-energy estimates. | Unit-contract approval, isolated credentials, pilot/full replay, label alignment, and REFIT anomaly evaluation. |

Table VI makes the evidence boundary operational. The four objectives are not presented as equally complete: Objectives 1 and 2 have implementation and selected local functional support, Objective 3 has the strongest completed offline evaluation, and Objective 4 has completed source and preprocessing work but no live replay. This mapping prevents the paper from converting a partially completed pipeline into a single undifferentiated success claim.

## 5. Limitations and Future Work

### 5.1 Historical Data Provenance and Generalisation

The legacy supervised CSV permits an exact local reconstruction, but the repository does not establish its external source, licence, acquisition conditions, physical units, household identity, or label-generation process. These absences limit scientific interpretation even when the model code and metrics are deterministic. A score obtained under an opaque data-generating process cannot be treated as a population estimate for household anomalies.

Avg_Past_Consumption is a second provenance concern. The code correctly computes ConsumptionDeviation from the supplied history field and avoids creating lag features from future rows, yet it cannot demonstrate whether the supplied history value was causal at the original point of prediction. This uncertainty matters because the model may benefit from a feature that would not be available under a live household deployment. Future work requires a documented causal feature construction before model transfer or retraining is claimed.

### 5.2 Simulator-Compatible Telemetry and Physical Measurement

Wokwi-compatible requests are useful for software integration demonstration because they exercise the application contract without requiring physical hardware in every test. They do not calibrate voltage, current, power factor, electrical noise, sampling error, clock drift, or meter installation. Consequently, the project does not make a utility-billing or physical-meter accuracy claim. Any physical validation would require a documented meter, reference instrument, sampling protocol, and comparison procedure.

### 5.3 REFIT Labels, Source Semantics, and Aggregate Evaluation

REFIT provides aggregate power and appliance-level channels, but the current paper does not transform its available annotation evidence into whole-house anomaly ground truth. The official anomaly resource is appliance-level and only House 1 overlaps with the H1/H2/H4 primary cohort; its detailed schema, timestamp semantics, count, and aggregate-alignment method were not verified in the present work. The REFIT Issues field is retained as context, not used as a label.

The interval-energy value is an actual-delta sample-hold estimate from Aggregate power in watts. It is deliberately not called a direct REFIT kWh measurement. The source reading semantics, possible treatment of a point sample versus interval average, and the mapping into the application's energyConsumed field still require review. A future evaluation must prevent both unit substitution (watts sent as kWh) and a second conversion of an already estimated interval energy.

### 5.4 Runtime Integration and Alert Correctness

The prepared REFIT cohort has not been provisioned as three isolated device identities. No H1/H2/H4 device-token mapping, live ingestion run, event reconciliation ledger, or full

chronological replay has been completed. Existing synthetic local alert and deduplication checks provide bounded evidence for the application, but they do not establish what will happen to a recorded household stream or to events whose timestamps span days rather than the local test window.

The current deduplication mechanism is also a policy rather than an exactly-once guarantee. It keys an alert by user, device/household scope, and type within a configured temporal window. It should therefore be evaluated for false suppression, repeated delivery, concurrent consumers, restarts, and database timing before being claimed as a reliable operational guarantee. A future replay must log both suppressed and delivered decisions alongside the corresponding source events.

**5.5 Resource and Deployment Constraints**

The documented machine has 8 GB RAM and a 3.696 GiB Docker allocation, while the main stack was observed at approximately 2.89 GiB. A separately isolated REFIT provisioning stack must therefore not run concurrently with the main stack on this workstation. This is a local experimental-control and safety constraint, not evidence that the architecture cannot run on hardware with appropriate resources.

**5.6 Performance, Resilience, and Security Evaluation**

The existing single-host k6 result, focused component tests, local alert checks, and limited fault observations are informative but insufficient for a broad operational claim. The k6 run did not measure distributed load, consumer lag, database saturation, or replay performance. The in-process model benchmark excludes HTTP, Kafka, storage, and alert overhead. Fault records do not prove post-restart reconnection or unique message processing. Security tests do not constitute an independent penetration test, mTLS deployment, secret rotation validation, or a production workload-identity review.

**5.7 Prioritised Future Validation Plan**

Stage 1 - Contract review: verify the semantic unit and aggregation meaning of energyConsumed against the derived_interval_energy_kwh field. Acceptance requires a documented mapping, payload validation, and no implicit or duplicate conversion. Stage 2 - Isolated provisioning: create a separate non-production identity environment with one profile, one aggregate-meter device, and one unique device credential for each of H1, H2, and H4. Acceptance requires redacted evidence that the mapping is one-to-one and does not reuse an existing household device.

Stage 3 - Pilot replay: transmit a bounded chronological sample with a dry-run log and conservative retry handling. Acceptance requires reconciliation of submitted, accepted, rejected, ambiguous, broker-consumed, and persisted records. The pilot should stop on authentication, schema, timestamp, or unit disagreement. Stage 4 - Label protocol: obtain and inspect the official annotation schema, define the target aggregate label, verify

timestamp alignment, and pre-register the temporal or household split before reporting any accuracy metric.

Stage 5 - Controlled full replay: replay the H1/H2/H4 primary cohort only after pilot approval, preserve original source time separately from replay time, and report all reconciliations. Stage 6 - Operational evaluation: use controlled duplicate, notification, failure, restart, and input-range scenarios to test alert behaviour; repeat latency and throughput measurements with stated hardware conditions. Each stage is an acceptance gate rather than an automatic continuation of the previous one.

## 6. Conclusion

This study documents a local, evidence-aware Home Energy Tracker with gateway-mediated admission, Kafka event processing, time-series and relational storage, deterministic insights, a legacy Random Forest inference service, a duplicate-aware alert path, and observability. The historical classifier was reproduced offline with its dataset-specific metrics, while a separate endpoint smoke test was correctly limited to interface execution. REFIT work established checksum-verified inputs, a defined H1/H2/H4 primary cohort, actual-delta interval-energy normalisation, quality controls, and deterministic reproduction of derived artefacts.

The contribution is therefore architectural traceability, historical-experiment reproducibility, and recorded-household data preparation—not real-household anomaly accuracy or end-to-end runtime reliability. Future validation must proceed through isolated provisioning, data-contract verification, a small pilot replay with event reconciliation, label-alignment review, and only then a controlled H1/H2/H4 replay with deduplication, recovery, and performance tests.

The four objectives are satisfied at different levels. The paper documents an implemented admission and event-processing architecture, including explicit user and device telemetry boundaries. It reconstructs the legacy supervised baseline exactly under the available historical CSV protocol. It also establishes checksum-verified and byte-reproducible REFIT preparation for a defined primary cohort. These outcomes are useful because they make the present state inspectable and reduce ambiguity about what the project has and has not shown.

The strongest empirical claims remain limited. The reported Random Forest metrics are historical CSV-specific results, the Wokwi-compatible response is an interface smoke test, the dated local system observations are single-host and configuration-specific, and REFIT has not been sent through the application or used for aggregate anomaly accuracy. The system should therefore be understood as a local implementation and reproducibility study rather than a completed real-household deployment.

The next credible advance is not another generic demonstration. It is a controlled, isolated REFIT evaluation with a verified energy contract, device-bound credentials, pilot reconciliation, a defensible label-alignment protocol, and measured alert, performance, and recovery outcomes. Completing those gates would allow future work to move from transparent preparation and historical reproducibility toward a substantiated household-level anomaly-detection claim.

## CRediT Authorship Contribution Statement

Rohit Shrivastava: Conceptualization, methodology, software, REFIT data preprocessing, formal analysis, visualisation, and writing—original draft. Harsh Gupta: Microservice implementation, Docker integration, validation and testing, and writing—review and editing. Anshika Rai: Literature review, data curation, documentation, result-table and visualisation preparation, and writing—review and editing. Syamasudha Veeragandham: Methodology review, validation, writing—review and editing, project coordination, and correspondence.

## Funding Statement

No external funding, grant, or other support was received for this work.

## Declaration of Competing Interests

The authors declare no competing financial interests or personal relationships that could have influenced the work reported in this paper.

## Data Availability

The cleaned REFIT data are identified through the dataset records in [6] and [7]. The historical labelled CSV and locally generated preprocessing artefacts are not declared as a public release in the available materials; their availability therefore cannot be guaranteed and remains subject to applicable source terms and author approval.

## References

[1] C. Rovetto, E. Cruz, A. Smolarz, D. Concepcion, E. E. Cano, and I. Nuñez, "Design of a microservices-based architecture for residential energy efficiency monitoring," International Journal of Electronics and Telecommunications, vol. 70, no. 4, pp. 1089–1098, 2024, doi: 10.24425/ijet.2024.152511.

[2] L. L. Motta, W. S. Alvarez, H. R. Carvajal, F. B. Neto, E. R. de Lima, and L. G. P. Meloni, “A cloud architecture for home energy management systems: a conceptual model,” Energy Informatics, vol. 8, Art. no. 142, 2025, doi: 10.1186/s42162-025-00599-1.

[3] A. F. S. Veloso, J. V. Reis, Jr., and R. A. L. Rabelo, “A Microservices-Based Solution with Hybrid Communication for Energy Management in Smart Grid Environments,” Sensors, vol. 26, no. 5, Art. no. 1714, 2026, doi: 10.3390/s26051714.

[4] Á. Hernández, R. Nieto, L. de Diego-Otón, M. C. Pérez-Rubio, J. M. Villadangos-Carrizo, D. Pizarro, and J. Ureña, “Detection of Anomalies in Daily Activities Using Data from Smart Meters,” Sensors, vol. 24, no. 2, Art. no. 515, 2024, doi: 10.3390/s24020515.

[5] D. Murray, L. Stankovic, and V. Stankovic, “An electrical load measurements dataset of United Kingdom households from a two-year longitudinal study,” Scientific Data, vol. 4, Art. no. 160122, 2017, doi: 10.1038/sdata.2016.122.

[6] D. Murray, L. Stankovic, and V. Stankovic, “REFIT: Electrical Load Measurements (Cleaned),” dataset, University of Strathclyde, Jun. 2016, doi: 10.15129/9ab14b0e-19ac-4279-938f-27f643078cec.

[7] D. Murray and L. Stankovic, “REFIT: Electrical Load Measurements (Cleaned),” Zenodo, dataset, Jun. 2016, doi: 10.5281/zenodo.5063428.

[8] K. Kulkarni, R. Dantakale, S. Mande, S. Kshatriya, and M. Agrawal, “Anomaly Detection in Smart Electric Meters for Detecting Faults and Misuse of Electric Energy Consumption,” International Journal of Scientific Research in Engineering and Management, vol. 8, no. 5, pp. 1–5, May 2024, doi: 10.55041/IJSREM34689.

[9] M. H. Abu-Arqoub, A. A. Titinchi, and R. R. Marie, “A Scalable IoT-AI Framework for Real-Time Energy Anomaly Detection in Smart Grid Networks,” in 2026 2nd International Conference on Computational Intelligence Approaches and Applications (ICCIAA), 2026, pp. 1–6, doi: 10.1109/ICCIAA68481.2026.11543846.

[10] S. Maitra, S. Kundu, and A. Shankar, “A Real-time Anomaly Detection Using Convolutional Autoencoder with Dynamic Threshold,” arXiv preprint arXiv:2404.04311, 2024, doi: 10.48550/arXiv.2404.04311.

[11] K. Karthika, T. C. G. Thrupthi Chandana, T. Sheikh, Sankeerthana, and Umme, “Smart Energy Monitoring System,” International Journal of Innovative Research in Information Security, vol. 11, no. 9, Oct. 2025, doi: 10.26562/ijiris.2025.v1109.15.

[12] J.-H. Liu and Y.-T. Hsiao, “Smart plug-based home energy management with IoT and anomaly detection,” Journal of the Chinese Institute of Engineers, vol. 49, no. 3, pp. 655–670, 2026, doi: 10.1080/02533839.2025.2557233.

[13] K. M. W. Hidayat and M. G. Al-Faris, “IoT-Based Electrical Power Consumption Monitoring System in Households Using ESP32 and PZEM-004T,” Brilliance: Research of Artificial Intelligence, vol. 5, no. 2, pp. 1077–1081, 2025, doi: 10.47709/brilliance.v5i2.6368.

[14] S. Sirisumrannukul, T. Intaraumnauy, and N. Piamvilai, “Optimal control of cooling management system for energy conservation in smart home with ANNs-PSO data analytics microservice platform,” Heliyon, vol. 10, no. 6, Art. no. e26937, 2024, doi: 10.1016/j.heliyon.2024.e26937.

[15] M. S. Hossain and M. M. Hassan, “Smart Energy Meter with Real Time Monitoring System,” in 2024 13th International Conference on Electrical and Computer Engineering (ICECE), 2024, pp. 761–766, doi: 10.1109/ICECE64886.2024.11024970.

[16] D. G. S, T. Gangakhedkar, A. P, A. Ashwija and C. B. T. Rambapuri, “IOT Enabled Electrical Consumption Tracking for Intelligent Energy Saving,” International Journal of Scientific Research in Engineering and Management, 2025. doi: 10.55041/ijsrem54847.

[17] Y. Zhu and M. Yue, “Design of an IoT-Based Smart City Energy Management System,” in 2025 4th International Conference on Green Energy and Power Systems (ICGEPS), 2025, pp. 489–492, doi: 10.1109/ICGEPS65133.2025.11034388.

[18] S. Susikala, R. K, J. K, S. K, Y. M, “IoT Based Smart Electricity Monitoring System with Real Time Alerts & Remote Control,” International Journal of Engineering & Extended Technologies Research, 2026. doi: 10.15662/ijeetr.2026.0802420 .

[19] A. S. Shuaibu, S. U. Haq, B. Almadani, F. Aliyu, and E. G. Al-Nahari, “Smart Metering Meets AI: Real-Time Appliance Monitoring and Anomaly Detection Over DDS Middleware,” IEEE Transactions on Industry Applications, vol. 62, no. 2, pp. 2326–2342, Mar. 2026, doi: 10.1109/TIA.2025.3608687.

[20] T. B. de Sousa and R. de A. L. Rabelo, “Smart Metering Enabled by the Internet of Things for Monitoring Residential Energy Consumption and Power Quality,” IEEE Access, vol. 13, pp. 186872–186886, 2025, doi: 10.1109/ACCESS.2025.3627191.